Ambient Email Agent

Operating Guide — how to set up, run, test, and extend this project on a new machine

Repository: **https://github.com/AryanNehra/ambient-email-agent** (public)
Built on LangChain Academy's course *Building Ambient Agents with LangGraph* (langchain-ai/agents-from-scratch, MIT License), extended with a prompt-injection guard (Extension A) and an LLM-judge calibration study (Extension B).

This PDF is a packaged, self-contained operating manual. The authoritative, always-up-to-date version of this information lives in README.md at the repository root — read that first if the two ever disagree.

1. What this project is

An email assistant built with LangGraph: an incoming email is triaged (ignore / notify / respond), a response agent drafts tool calls (write a reply, check calendar, schedule a meeting), every send/schedule action pauses for human-in-the-loop (HITL) approval, and approved feedback can update long-term memory. All of that is course code, unmodified except for one documented line in tests/conftest.py (see Section 7).

Two original extensions were added on top:

- **Extension A — prompt-injection guard** (src/email_assistant/security/): a 4-layer, independently-toggleable guard (content framing, a deterministic tool-call validator, a heuristic injection detector, and a memory-write guard), evaluated against a 24-case attack/benign dataset. Offline result: 17/17 attacks blocked, 0/7 benign false positives, with the full guard on.
- **Extension B — LLM-judge calibration** (evals/judge_calibration/): measures how much the course's existing LLM-as-judge agrees with a real human on response quality, across 30 items. Requires a human to actually label the items (see Section 6) — the analysis script refuses to invent numbers if labels or a live API key are missing.

2. Prerequisites on a new machine

- Python 3.11 or 3.12 (the course pins >=3.11,<3.14 in pyproject.toml — Python 3.14 will NOT work; check with `python3 --version` first).
- git
- GitHub CLI (gh), optional, only needed if you want to push further changes from the command line instead of plain git.
- An OpenAI API key and a LangSmith API key — **optional**. Everything this project's own test suite and offline evals need runs with NO key at all. A real key is only needed for: the course's own live tests (tests/run_all_tests.py), live-mode guard evaluation, and the judge-calibration analysis.

3. Getting the code onto the new machine

Option A — clone from GitHub (recommended, always up to date):

```
git clone https://github.com/AryanNehra/ambient-email-agent.git
cd ambient-email-agent
```

Option B — unzip the transferred archive:

```
unzip ambient-email-agent.zip
cd ambient-email-agent
```

The zip includes the full git history (.git/), so even if you unzip it you can still run git log, git diff, etc. It does NOT include the Python virtual environment (.venv) — that's machine-specific and gets recreated in the next step.

4. Environment setup

```
python3.12 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -e ".[dev]"
```

This installs the course package (importable as `email_assistant`) plus dev tools (ruff, mypy, pytest).

4.1 Optional: API keys for live mode

```
cp .env.example .env
# then edit .env and fill in:
# OPENAI_API_KEY=...
# LANGSMITH_API_KEY=...
# LANGSMITH_TRACING=true
```

Never commit a real .env file — it's already in .gitignore. The repo's own offline tests set a harmless placeholder key automatically (see Section 7); you only need a real key for the live-mode commands below.

5. Day-to-day commands (Makefile)

Command	What it does
`make install`	pip install -e ".[dev]"
`make lint`	ruff check on the code added in this project (security/, evals/, tests/security, tests/judge_calibration)
`make test`	92 offline tests, no API key needed, a few seconds
`make eval-injection`	runs the Extension A offline eval, writes results/injection_summary.{json,md}
`make eval-calibration`	runs the Extension B analysis (writes 'pending' unless labels + a real key exist)
`make label`	launches the interactive judge-calibration labeling CLI

These all assume the venv is activated (`source .venv/bin/activate`) and `OPENAI_API_KEY` is unset or a placeholder for the offline ones.

5.1 Course's own live tests and notebooks

These need a real OpenAI + LangSmith key (Section 4.1):

```
python tests/run_all_tests.py # course's own agent test suite
pytest tests/test_notebooks.py -v # verify notebooks execute
python -m evals.injection.run --live --repeats 3 # guard eval against a real model
```

6. Completing the judge-calibration study (Extension B)

As of the last push, this is the only piece left **pending** and requires a real human (you) — it cannot be automated or faked without defeating its purpose.

- **Label the 30 items** (~20-30 minutes):

```
python -m evals.judge_calibration.label
```

Shows each item's email, the agent's response, and the rubric, one at a time, in a randomized order (so you aren't anchored by the original authoring order). Commands: `p[ass] [1-5]`, `f[ail] [1-5]`, s(kip), u(ndo), q(uit — saves progress, resume later by running the same command again). A CSV fallback exists: `python -m evals.judge_calibration.label --csv-template`, fill it in a spreadsheet, then `python -m evals.judge_calibration.label --from-csv`.

- **Run the real analysis** (needs a real OPENAI_API_KEY in .env):

```
python -m evals.judge_calibration.analyze
```

Writes results/judge_calibration.{json,md}: accuracy vs. your labels, Cohen's kappa, confusion matrix, false-pass/false-fail rates, judge self-consistency across 3 repeats, a verbosity-bias probe, and a rubric-paraphrase-wording sensitivity probe, with a 95% bootstrap confidence interval (n=30 is small — the report says so explicitly).

After this, update the "Extension B" section of README.md to replace the "Status: pending" paragraph with the real numbers from results/judge_calibration.md — copy them, don't retype from memory.

7. The only two changes made to inherited course code

File	Change	Why
tests/conftest.py	One line sets a placeholder OPENAI_API_KEY if unset	So several course assistant modules construct a ChatOpenAI client at import t
.github/workflows/*.yml	Removed the course's 2 Claude-Code-Action workflows	They required an ANTHROPIC_API_KEY secret that isn't configured here

No other course file was changed. Full attribution lives in NOTICE.md.

8. Git and GitHub workflow

Every change so far was made on a short-lived feature branch and merged into `main` with `git merge --no-ff` (so the history keeps a visible merge commit per logical change), then the feature branch was deleted. To make a further change the same way:

```
git checkout main && git pull
git checkout -b feat/your-change-name
# ... make changes, then:
git add -A && git commit -m "feat: describe the change"
git checkout main
git merge --no-ff feat/your-change-name -m "Merge feat/your-change-name: ..."
git branch -d feat/your-change-name
git push
```

8.1 GitHub authentication on a new machine

The repo is owned by the GitHub account **AryanNehra**. On a new machine you'll need to authenticate gh (and git's credential helper) as that account before you can push:

```
gh auth login --hostname github.com --web
# choose "Login with a web browser", sign in as AryanNehra
gh auth setup-git
```

Known gotcha: if your global git config has a URL rewrite rule that forces https://github.com/ URLs to SSH (check with `git config --global --get-regexp '^url\.'`), it can silently route pushes through an SSH key tied to a *different* GitHub account, causing a confusing "permission denied to <other-account>" error even though gh shows you're logged in as the right one. Fix, from inside the repo:

```
git remote set-url origin https://github.com/AryanNehra/ambient-email-agent.git
git config --local url."https://github.com/AryanNehra/".insteadOf
"https://github.com/AryanNehra/"
# the second line adds a MORE SPECIFIC local rule that overrides the
# global SSH rewrite, just for this account's URLs, just in this repo
```

9. Where things live

Path	What's there
`src/email_assistant/`	Course code (agent, tools, prompts, eval helpers) — unmodified except conftest.py note above
`src/email_assistant/security/`	Mine — the Extension A guard (framing.py, tool_validator.py, detector.py, memory_guard.py, guard.py, decision_log.py, config.py)
`evals/injection/`	Mine — Extension A: cases.jsonl (24 cases), run.py (offline/live eval runner), evaluator.py, dataset.py
`evals/judge_calibration/`	Mine — Extension B: items.jsonl (30 items), label.py (labeling CLI), judge.py, metrics.py, analyze.py
`tests/security/`, `tests/judge_calibration/`	Mine — 92 offline tests
`docs/threat-model.md`	Mine — the threat model behind Extension A
`docs/judge-calibration-methodology.md`	Mine — honest account of how the 30 Extension B items were built
`results/`	Mine — generated, committed result summaries (injection_summary.*, judge_calibration.*)
`notebooks/`, `tests/test_response.py`, `tests/run_all_tests.py`	Course code, unmodified
`.github/workflows/ci.yml`	Mine — CI (lint + offline tests, no secrets)
`NOTICE.md, README.md`	Attribution and the full project writeup — read README.md first

10. Known limitations (be honest about these in interviews)

- The guard is implemented, unit-tested, and evaluated, but is **not yet wired into the live graph nodes** in email_assistant_hitl.py — it runs alongside the graph in tests and the eval harness. See README's Limitations section for the exact two call sites where wiring it in would go.

- The heuristic detector is regex-based and bypassable by a sufficiently creative attacker — it's a deterministic floor, not a guarantee.
- 17 attack cases is not exhaustive. n=30 for judge calibration is a small sample; treat point estimates together with their 95% CI.
- All tools (email, calendar) are mocks; the real Gmail integration was not touched or evaluated by either extension.